

AI Report

We are highly confident this text is

AI Generated

AI Probability

80%

This number is the probability that the document is AI generated, not a percentage of AI text in the document.

Plagiarism



The plagiarism scan was not run for this document. Go to gptzero.me to check for plagiarism.

LAPORAN TUGAS 1 FAGIL LOLA P (2410651031).pdf - 10/19/2025

LAPORAN TUGAS 1-4 SISTEM OPERASI Dosen Pengampu: Triawan Adi Cahyanto, M.Kom Disusun Oleh: Fagil Lola Prihernando (2410651031) PRODI TEKNIK INFORMTIKA FAKULTAS TEKNIK UNIVERSITAS MUHAMMADIYAH JEMBER 2025 BAB 1 PENDAHULUAN LATAR BELAKANG Perkembangan teknologi sistem operasi modern menuntut pemahaman mendalam tentang konsep manajemen proses, manajemen file, shell scripting, dan monitoring sistem. Dalam lingkungan komputasi kontemporer, kemampuan untuk mengembangkan tools yang dapat memantau, mengelola, dan berinteraksi dengan sistem operasi menjadi kompetensi fundamental bagi mahasiswa teknik informatika. Praktikum sistem operasi ini dirancang untuk memberikan pengalaman langsung dalam mengimplementasikan berbagai konsep teoritis ke dalam aplikasi praktis melalui empat tugas utama yang mencakup aspek-aspek kritis dalam pengelolaan sistem. Implementasi process manager, file monitor, custom shell, dan system information tool tidak hanya memperkuat pemahaman konseptual tetapi juga mengembangkan keterampilan pemecahan masalah dalam lingkungan sistem yang sesungguhnya. Melalui pendekatan hands-on ini, mahasiswa dapat memahami kompleksitas dan mekanisme internal sistem operasi secara komprehensif. RUMUSAN MASALAH 1. Bagaimana mengimplementasikan manajemen proses yang efektif menggunakan fork, pipe, dan shared memory untuk menangani eksekusi paralel multiple processes? 2. Bagaimana membangun sistem monitoring file yang mampu mendeteksi perubahan secara real-time dan mencatat aktivitas filesystem dengan notifikasi yang komprehensif? 3. Bagaimana mengembangkan custom shell dengan fitur advanced seperti multiple pipes, redirection, dan background processes yang stabil tanpa memory leaks? TUJUAN 1. Mengembangkan process manager yang dapat menangani eksekusi paralel tiga proses anak (faktorial, bilangan prima, dan pembalik string) dengan mekanisme IPC yang tepat. 2. Membuat file monitoring system yang mampu mendeteksi event CREATE, MODIFY, DELETE secara real-time dengan logging dan notifikasi yang andal. 3. Membangun custom shell dengan dukungan multiple pipes, I/O redirection, dan background processes yang memiliki manajemen memori yang optimal. BAB 2 PEMBAHASAN 2.1 TUGAS 1 Kode pada gambar menunjukkan proses kompilasi dan eksekusi program process_manager.c. Saat dikompilasi, muncul satu warning karena ketidaksesuaian format pada fungsi printf, yaitu %ld digunakan untuk mencetak nilai bertipe int. Setelah diperbaiki dengan perintah gcc -o process_manager process_manager.c, program berhasil dijalankan. Hasilnya menampilkan tiga proses anak: menghitung faktorial, menampilkan bilangan prima, dan membalik string sesuai masukan pengguna. 2.1.2 Penjelasan Proses Komplikasi dan Eksekusi 1. cd Praktikum_os Fungsi: Masuk ke direktori bernama

praktikum_os. Tujuan: Agar kamu bisa mengakses file sumber program process_manager.c yang berada di folder tersebut. Penjelasan tambahan: cd (change directory) digunakan untuk berpindah direktori kerja di terminal Linux.

2. gcc process_manager process_manager.c Fungsi: Mencoba mengompilasi file process_manager.c dan menghasilkan file biner bernama process_manager. Kesalahan: o warning: format '%ld' expects argument of type 'long int', but argument 4 has type 'int' Terjadi karena %ld digunakan untuk menampilkan long int, tetapi variabel (int)usec bertipe int. o Harusnya gunakan %d jika variabel bertipe int. Error tambahan: o /usr/bin/ld: process_manager: stdin: invalid version 3 (max 0) o collect2: error: ld returned 1 exit status Menunjukkan proses linking gagal (biasanya karena kesalahan versi format biner atau parameter output salah).

3. ./process_manager Fungsi: Menjalankan file hasil kompilasi bernama process_manager. Tanda.: Menunjukkan bahwa file eksekusi berada di direktori saat ini (.). Hasil saat dijalankan: o Program meminta tiga input: 1. Angka untuk faktorial (Masukkan angka untuk faktorial: 5) 2. Batas bilangan prima (Masukkan batas bilangan prima: 6) 3. String untuk dibalik (Masukkan string untuk dibalik: ollah) o Output yang dihasilkan menunjukkan proses paralel (tiap child process memiliki PID berbeda): o Child PID: 1633 " Waktu: 0.749 detik " Hasil faktorial: 120 o Child PID: 1634 " Waktu: 0.420 detik " Bilangan prima: 2 3 5 o Child PID: 1635 " Waktu: 0.269 detik " String terbalik: hallo

4. nano Fungsi: Membuka teks editor terminal bernama Nano untuk membuat atau mengedit file teks (seperti process_manager.c). Contoh penggunaan: nano process_manager.c kode: 2.2 PRAKTIK TUGAS 2 2.2.1 File Monitor menjelaskan proses pembuatan, kompilasi, dan eksekusi program pemantauan direktori. Tahapan dimulai dengan membuat folder kerja menggunakan mkdir, lalu menulis kode file_monitor.c melalui nano. Setelah dikompilasi dengan gcc, program dijalankan untuk memantau direktori monitor_dir. Saat file dibuat, diubah, atau dihapus, program menampilkan notifikasi event beserta PID. Proses dihentikan menggunakan sinyal SIGTERM atau perintah kill.

Before (sebelum): Penjelasan Proses Kompilasi dan Eksekusi:

1. Persiapan Direktori: - User membuat direktori baru bernama 'file_monitor_task' menggunakan perintah 'mkdir'.
2. Mencoba Menjalankan Program: - User mencoba menjalankan perintah 'monitor_dir' tetapi gagal karena program tersebut belum tersedia (command not found).
3. Pembuatan Direktori Monitoring: - User membuat subdirektori 'monitor_dir' di dalam 'file_monitor_task' untuk keperluan monitoring file.
4. Pengembangan Program: - User membuat file source code C bernama 'file_monitor.c' menggunakan perintah 'touch'.
5. Proses Kompilasi: - User mengompilasi program C menggunakan compiler GCC dengan sintaks: 'gcc -o file_monitor file_monitor.c' - Flag '-o' digunakan untuk menentukan nama output executable (file_monitor).
6. Menjalankan Program: - User mengeksekusi program dengan perintah: './file_monitor./monitor_dir' - Program berhasil dijalankan dengan parameter direktori yang akan dimonitor ('./monitor_dir').
7. Output Program: - Program menampilkan pesan bahwa File Monitor telah berjalan.
8. Verifikasi Direktori: - User membuka terminal lain dan masuk ke direktori 'monitor_dir'.

Menunjukkan direktori yang dimonitor: './monitor_dir' - Menampilkan PID (Process ID) program: 1812 - Menginformasikan file log yang digunakan: 'file_monitor.log' - Program terus berjalan dalam background hingga dihentikan dengan SIGTERM atau Ctrl+C.

8. Testing (uji coba): Penjelasan Proses Monitoring File:

1. Program Monitoring Berjalan: - Program 'file_monitor' telah aktif dengan PID 1862 - Memantau direktori './monitor_dir' - Menggunakan file log 'file_monitor.log' untuk pencatatan aktivitas
2. Aktivitas File yang Dimonitor: Pembuatan File: - User membuat file 'test1.txt' dengan perintah 'touch' - Program mendeteksi event CREATED dan mencatatnya di log - Mengirim notifikasi email dummy untuk event creation Modifikasi File: - User menambahkan konten "hello" ke 'test1.txt' dengan 'echo "hello" >> test1.txt' - Program mendeteksi event MODIFIED - Mengirim notifikasi email dummy untuk event modification
3. Pembuatan File Kedua: - User membuat file 'file2.log' dengan perintah 'touch' - Program mendeteksi event CREATED untuk file baru tersebut - Mencatat aktivitas dalam log file
4. Penghapusan File: - User menghapus 'test1.txt' dengan perintah 'rm' - Program mendeteksi event DELETED - Mengirim notifikasi email dummy untuk event deletion

3. Format Notifikasi: - Setiap event menampilkan notifikasi email dummy dengan

format konsisten - Informasi yang dicakup: penerima, subjek, dan pesan alert - Log mencatat: timestamp, jenis event, nama file, dan PID proses

4. Fungsi Program yang Terdemonstrasi:

- Real-time Monitoring: Mendeteksi perubahan file secara langsung
- Multiple Event Types: Membedakan antara CREATE, MODIFY, dan DELETE
- Logging System: Mencatat semua aktivitas dengan timestamp
- Notification Simulator: Menampilkan notifikasi email untuk setiap event
- Background Operation: Berjalan terus menerus hingga dihentikan manual

5. Alur Kerja:

- Program monitoring Deteksi perubahan file Catat di log Tampilkan notifikasi
- Semua operasi file oleh user terekam dan dilaporkan secara real-time
- Program tetap berjalan di background sambil memantau direktori target

After (setelah) Penjelasan Proses Penghentian dan Verifikasi Program:

1. Penghentian Program Monitoring:

- User menghentikan program monitoring dengan perintah `kill 1862`
- PID 1862 adalah proses ID yang sebelumnya ditampilkan saat program dijalankan
- Perintah `kill` mengirim sinyal SIGTERM (termination) secara default

2. Proses Shutdown Program:

- Program menerima sinyal SIGTERM dan memulai proses shutdown
- Menampilkan pesan: "Menerima signal SIGTERM, melakukan cleanup..."
- Melakukan prosedur cleanup untuk membersihkan resources
- Menampilkan konfirmasi: "Cleanup selesai. Program berhenti."
- Program keluar dengan normal dan kontrol kembali ke shell

3. Verifikasi File Log:

- User memeriksa isi file log dengan perintah `cat ~/file_monitor_task/file_monitor.log`
- File log berisi catatan semua event yang terdeteksi selama monitoring:
- CREATE untuk file `test1.txt` (23:06:10)
- MODIFIED untuk file `test1.txt` (23:07:15)
- CREATE untuk file `file2.log` (23:07:36)
- DELETED untuk file `test1.txt` (23:07:48)

4. Verifikasi Status Direktori:

- User memeriksa isi direktori `monitor_dir` dengan `ls -la`
- Hasil menunjukkan:
- Direktori current (.) dan parent (..) - File `file2.log` masih ada (0 bytes) - file yang dibuat selama monitoring
- File `test1.txt` sudah tidak ada - sesuai dengan log event DELETED

5. Konfirmasi Keberhasilan Sistem:

- Logging System: Berfungsi dengan baik, semua event terekam

● Sentences that are likely AI-generated.

FAQs

What is GPTZero?

GPTZero is the leading AI detector for checking whether a document was written by a large language model such as ChatGPT. GPTZero detects AI on sentence, paragraph, and document level. Our model was trained on a large, diverse corpus of human-written and AI-generated text with support for English, Spanish, French, German, and other languages. To date, GPTZero has served over 10 million users around the world, and works with over 100 organizations in education, hiring, publishing, legal, and more.

When should I use GPTZero?

Our users have seen the use of AI-generated text proliferate into education, certification, hiring and recruitment, social writing platforms, disinformation, and beyond. We've created GPTZero as a tool to highlight the possible use of AI in writing text. In particular, we focus on classifying AI use in prose. Overall, our classifier is intended to be used to flag situations in which a conversation can be started (for example, between educators and students) to drive further inquiry and spread awareness of the risks of using AI in written work.

Does GPTZero only detect ChatGPT outputs?

No, GPTZero works robustly across a range of AI language models, including but not limited to ChatGPT, GPT-5, GPT-4, GPT-3, Gemini, Claude, and AI services based on those models.

What are the limitations of the classifier?

The nature of AI-generated content is changing constantly. As such, these results should not be used to punish students. We recommend educators to use our behind-the-scenes [Writing Reports](#) as part of a holistic assessment of student work. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Instead, we recommend educators take approaches that give students the opportunity to demonstrate their understanding in a controlled environment and craft assignments that cannot be solved with AI. Our classifier is not trained to identify AI-generated text after it has been heavily modified after generation (although we estimate this is a minority of the uses for AI-generation at the moment). Currently, our classifier can sometimes flag other machine-generated or highly procedural text as AI-generated, and as such, should be used on more descriptive portions of text.

I'm an educator who has found AI-generated text by my students. What do I do?

Firstly, at GPTZero, we don't believe that any AI detector is perfect. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Nonetheless, we recommend that educators can do the following when they get a positive detection: Ask students to demonstrate their understanding in a controlled environment, whether that is through an in-person assessment, or through an editor that can track their edit history (for instance, using our [Writing Reports](#) through Google Docs). Check out our list of [several recommendations](#) on types of assignments that are difficult to solve with AI.

Ask the student if they can produce artifacts of their writing process, whether it is drafts, revision histories, or brainstorming notes. For example, if the editor they used to write the text has an edit history (such as Google Docs), and it was typed out with several edits over a reasonable period of time, it is likely the student work is authentic. You can use GPTZero's Writing Reports to replay the student's writing process, and view signals that indicate the authenticity of the work.

See if there is a history of AI-generated text in the student's work. We recommend looking for a long-term pattern of AI use, as opposed to a single instance, in order to determine whether the student is using AI.